

Implementacja modelu LLM+MCP z bazą danych

Projekt z przedmiotu Hurtownie Danych i Systemy Eksploracji Danych

08.06.2026

Michał Kubina,

Dominika Zakrzewska,

Grupa BDiISD2

1. Cel projektu

Celem projektu była integracja dużego modelu językowego (LLM) z dwiema relacyjnymi bazami danych za pośrednictwem MCP (Model Context Protocol). Dzięki temu model językowy otrzymuje bezpośredni dostęp do danych biznesowych i jednocześnie potrafi tworzyć trwałą pamięć w celu zachowywania kontekstu konwersacji i preferencji użytkownika pomiędzy sesjami.

2. Zakres wykonywanych prac

W ramach realizacji projektu wykonano następujące kroki:

- Skonfigurowano relacyjną bazę danych PostgreSQL, przechowującą dane biznesowe.
- Skonfigurowano lokalną bazę danych SQLite do obsługi pamięci agenta (przechowywania kontekstu oraz preferencji użytkownika).
- Utworzono środowisko oparte na Docker, automatyzujące wdrażanie baz danych oraz MCP.
- Przygotowano wirtualne środowisko Pythona (venv), w celu izolacji środowiska i zapobiegania konfliktów zależności w systemie.
- Stworzono dwa niezależne MCP: Business MCP – dane biznesowe oraz Memory MCP – pamięć, kontekst, preferencje.
- Zintegrowano środowisko OpenCode i skonfigurowano go do wykorzystywania stworzonych przez nas MCP oraz opcjonalnie agenta IBM granite 4.1:8b.

3. Dane wejściowe

Dane biznesowe

Struktura tabel oraz przykładowe rekordy danych biznesowych są inicjalizowane podczas startu kontenera. Dane te mają format relacyjny.

Dane kontekstowe

Dane kontekstowe są generowane dynamicznie w trakcie działania programu. Pochodzą bezpośrednio z konwersacji użytkownika z modelem LLM. Dana treść (content) jest zapisywana pod danym tematem (topic) wraz z automatycznie nadawanym czasem wygaśnięcia (`expires_at = now() + ttl_hours`). Podczas odczytu wybierane są tylko treści „nieprzeterminowane”, a podczas czyszczenia te wpisy, które są już „przeterminowane” (`expires_at > now()`).

4. Założenia projektowe

Wymagania

Do uruchomienia i prawidłowego działania projektu potrzebne są:

- Docker Desktop,
- Środowisko OpenCode,
- Python 3.12 lub nowszy,
- Plik dump (.sql) inicjalizujący bazę PostgreSQL,
- Opcjonalnie model IBM granite 4.1:8b uruchomiony za pomocą Ollama.

Użycie niebezpiecznych operacji

W projekcie umożliwiono agentowi wykonywanie zapytań DROP, DELETE, ALTER oraz TRUNCATE. Funkcjonalność ta została zaimplementowana w celu pokazania pełnych możliwości takiego rozwiązania. W rozwiązaniach komercyjnych nie powinny one być używane bez odpowiednich zabezpieczeń.

W naszym wypadku, w celu wprowadzenia minimalnych zabezpieczeń, LLM może wykonać te zapytania dopiero po przedstawieniu użytkownikowi dokładnego zapytania jakie chce wykonać oraz po otrzymaniu jasnego potwierdzenia („tak” lub „confirm” napisane przez użytkownika).

5. Opis rozwiązania

Głównym interfejsem użytkownika jest terminal środowiska OpenCode, które komunikują się z lokalnym lub oferowanym przez środowisko modelem LLM.

Kiedy użytkownik zadaje pytanie, model rozpoczyna od analizy intencji, a następnie sprawdza kontekst dla odpowiedniego tematu, aby przypomnieć sobie ustalenia czy preferencje z poprzednich rozmów.

Kolejnym krokiem jest wybór odpowiednich narzędzi. Jeżeli jest to zapytanie analityczne odnośnie bazy danych, korzysta z narzędzi dostępnych w Business MCP, a jeżeli dotyczy to zapamiętywania lub odczytu kontekstu, sięga po narzędzia dostępne w Memory MCP.

Oba serwery MCP działają jako osobne procesy w kontenerach Docker, a dane po zapytaniach zostają zapisane w odpowiednio biznesowej bazie danych lub kontekstowej bazie danych.

6. Model danych i baza danych

Baza biznesowa (PostgreSQL)

Struktura tej bazy jest dynamiczna i zależy od pliku inicjalizującego. Model poznaje jej schemat za pomocą narzędzia `get_schema`, który zwraca mapę tabel, kolumn, typów danych oraz relacji kluczy obcych.

Nasza przykładowa baza składa się z dwóch tabel:

- `customers`: `id`, `name`, `email`, `created_at`
- `transactions`: `id`, `customer_id`, `amount`, `transaction_date`

Baza pamięci kontekstowej (SQLite)

Struktura jest stała i opiera się na jednej tabeli `context`, składającej się z:

- `id`,
- `topic` – kategoria/temat,
- `content` – treść przechowywanego faktu/informacji,
- `created_at` – czas utworzenia,
- `expires_at` – czas wygaśnięcia.

7. Implementacja i walidacje

Kod

Business MCP – Posiada funkcję dla asystenta AI oznaczane adnotacją `@mcp_tool()` służące jako narzędzia umożliwiające mu dostęp do bazy danych i jej modyfikację, a także do pobierania schematu bazy danych, co umożliwi mu ich pobieranie. Każda taka funkcja ma opis na jej początku, który służy do poinformowania agenta o możliwościach danego narzędzia. Wykorzystany został on również do wprowadzenia zabezpieczeń w formie dodatkowego zapytania dla narzędzi modyfikujących dane. Oprócz tego w pliku mamy też funkcję tworzącą (`_get_pool()`) i zamykającą (`cleanup()`) połączenie z bazą.

Memory MCP – Tutaj także zostały wykorzystane `@mcp_tools()` do stworzenia narzędzi umożliwiających zapisywanie kontekstu (`save_context()`), pobieranie kontekstu (`retrieve_context()`), czyszczenia przestarzałego kontekstu (`cleanup_expired_context()`), wypisanie tematów kontekstu (`list_topics()`). Dodatkowo znowu są funkcje łączące się z bazą (`_get_connection()`), tworzące nową tabelę jeżeli nie została jeszcze stworzona (`_initialize_database()`) i zamykające połączenie (`cleanup()`).

Biblioteki

Projekt zaimplementowano w języku Python, korzystając między innymi z bibliotek:

- `fastmcp` – do obsługi protokołów,
- `asyncpg` – do połączeń asynchronicznych z PostgreSQL,
- `sqlite3` – do obsługi bazy danych SQLite,
- `dotenv` – do obsługi zmiennych środowiskowych.

Konfiguracja i uruchomienie

Aby skorzystać z rozwiązania na początku należy skonfigurować środowisko poprzez wywołanie podanych niżej poleceń w terminalu otworzonym w głównym folderze projektu.

1. Prepare the virtual environment:

```
python3 -m venv .venv  
source .venv/bin/activate
```
2. Install dependencies:

```
pip install -r requirements.txt
```
3. Prepare environment variables:

```
cp .env.example .env
```

Opcjonalnie można edytować plik `docker-compose.yml` i zmodyfikować ścieżkę instalacji (mount path) `.sql` jeżeli jest to potrzebne.

```
./init-db.sql:/docker-entrypoint-initdb.d/init.sql
```

Po modyfikacji pliku `.env` do swoich potrzeb (np. zmianie hasła bazy danych, hostnames) można uruchomić kontenery oraz agenta LLM.

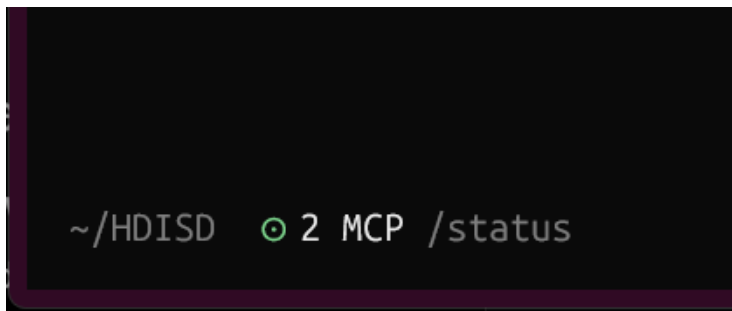
1. Start the database in the background

```
docker compose up -d
```

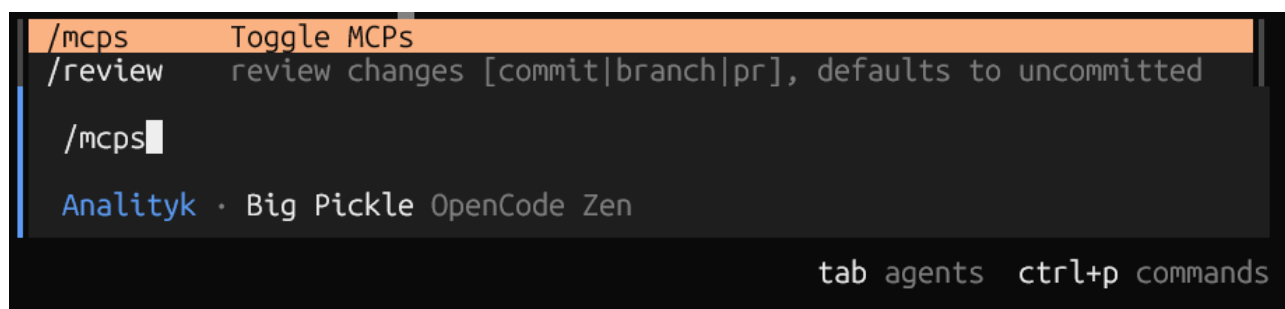
2. Start the OpenCode agent
opencode

Pierwsze kroki

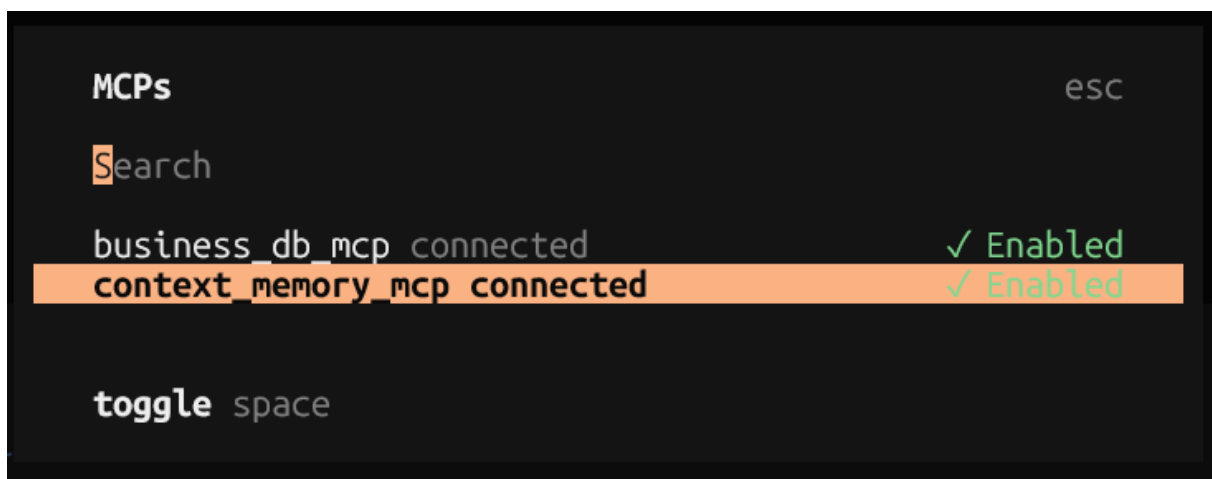
Po włączeniu OpenCode w lewym dolnym rogu widać kropkę wraz z liczbą oraz napisem MCP. Jest to informacja o poprawnym połączeniu się MCP do LLM. Jeżeli wszystko działa poprawnie kropka powinna zapalić się na zielono a numer zmienić na 2 (Rys. 1.). Możemy też sprawdzić gotowość MCP wpisując „/mcps” w czat OpenCode (Rys. 2., Rys. 3.).



Rys. 1. Informacja o poprawnym połączeniu z MCP w OpenCode.

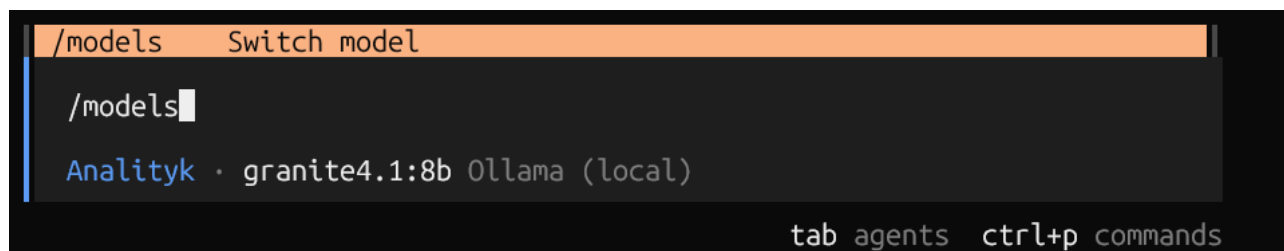


Rys. 2. Komenda do sprawdzenia połączenia z MCP w OpenCode.

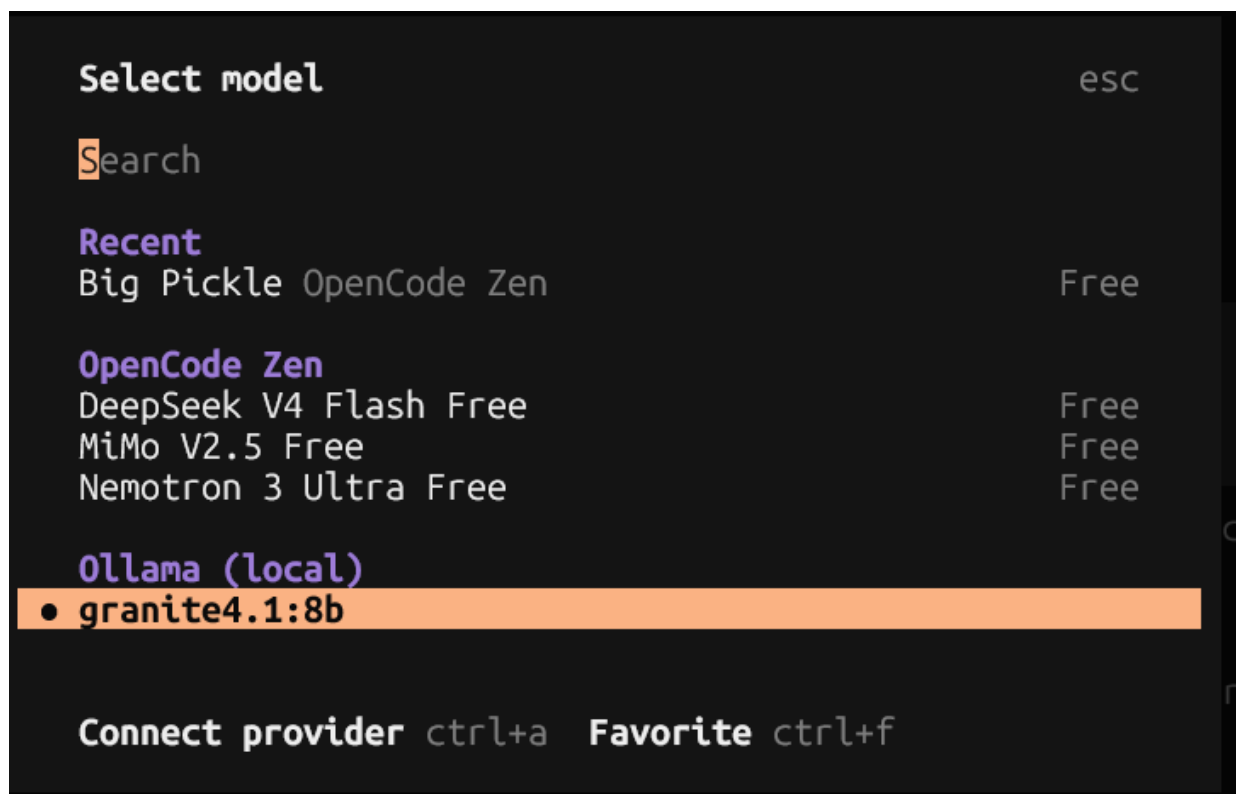


Rys. 3. Widok po wpisaniu komendy /models.

Możemy także zmienić model naszego agenta poprzez wpisanie „/models” w czat (Rys. 4.). Do wyboru będą darmowe agenty oferowane przez OpenCode oraz agent IBM granite 4.1:8b jeżeli został on zainstalowany przez użytkownika (Rys. 5.).



Rys. 4. Komenda do zmiany modelu agenta w OpenCode.



Rys. 5. Widok po wpisaniu komendy /models.

Walidacja poprawności działania

W celu przetestowania poprawności działania programu wysłano listę 18 zapytań i sprawdzono poprawność otrzymanych odpowiedzi.

1. "Describe the database structure to me. What tables are there and how are they related?"
2. "What columns does the customers table contain, and can the email field accept NULL values?"
3. "Fetch all transactions from the transactions table that have an amount grater than 100."
4. "Calculate the total amount spent by each customer using the transactions table. "
5. "Show me the names of all customers along with the amount of their transactions by joining the customers and transactions tables. "

6. "Add a new customer to the customers table with the name Alice Brown and email alice@example.com."
7. "Update the email of the customer named John Doe to john.doe@newdomain.com. "
8. "Increase the amount of all transactions below 100 by 10% in the transactions table. "
9. "Create a new table named support_tickets. Add the following columns: id (integer), customer_id (integer), issue (text), and status (varchar). "
10. "Add a new phone_number column of type VARCHAR to the existing customers table."
11. "Delete all entries from the transactions table where the amount is less than 80."
12. "Delete the customer from the customers table whose email is jane@example.com."
13. "Completely clear the transactions table."
14. "Completely drop the newly created support_tickets table from the database."
15. "Remember to always end your respons with a seperator of dots. Save this for 48 hours."
16. "What formatting preferences have you saved in the context?"
17. "Clean up the memory and remove all expired entries."
18. "Check our contextual memory. What topics are you currently tracking? "

8. Analiza wyników

Poniżej są odpowiedzi agenta na 3 różne pytania z listy testowej.

```
Describe the database structure to me. What tables are there and how are they related?

+ Thought: 922ms
🔧context_memory_mcp_retrieve_context [topic=database_structure]
🔧business_db_mcp_get_current_user_privileges
🔧business_db_mcp_get_schema [schema_name=public]

+ Thought: 1.1s

Świetnie! Oto struktura bazy danych, którą znalazłem:

👤 Konto i uprawnienia

- Użytkownik: postgres (superużytkownik PostgreSQL)
- Uprawnienia: Pełny dostęp (INSERT, SELECT, UPDATE, DELETE, TRUNCATE, REFERENCES, TRIGGER) do wszystkich tabel.

🗄️ Baza danych – schemat public

Baza składa się z 2 tabel powiązanych relacją klucza obcego:

1. Tabela: customers (klienci)



| Kolumna    | Typ       | Ograniczenia            |
|------------|-----------|-------------------------|
| id         | INTEGER   | NOT NULL (klucz główny) |
| name       | VARCHAR   | NOT NULL                |
| email      | VARCHAR   | NOT NULL                |
| created_at | TIMESTAMP | nullable                |



2. Tabela: transactions (transakcje)
```

Rys. 6. Część pierwsza odpowiedzi na pytanie 1.

2. Tabela: transactions (transakcje)

Kolumna	Typ	Ograniczenia
id	INTEGER	NOT NULL (klucz główny)
customer_id	INTEGER	nullable → klucz obcy do customers(id)
amount	NUMERIC	NOT NULL
transaction_date	TIMESTAMP	nullable

🔗 Relacje między tabelami

customers (1) — (N) transactions

- Relacja **jeden-do-wielu**: jeden klient może mieć wiele transakcji.
- Połączenie odbywa się przez kolumnę `transactions.customer_id`, która jest **kluczem obcym** wskazującym na `customers.id`.
- `customer_id` może być NULL (dopuszczalne są transakcje nieprzypisane do żadnego klienta).

Podsumowanie

Jest to prosty, klasyczny schemat dla systemu obsługi klientów z transakcjami. Jeśli chcesz, mogę:

- Pobrać przykładowe dane z tych tabel,
- Wyświetlić listę klientów i ich transakcje za pomocą JOIN,
- Dodać nową tabelę,
- Zapisać tę strukturę do kontekstu pamięci na przyszłość.

Rys. 7. Część druga odpowiedzi na pytanie 1.

Calculate the total amount spent by each customer using the transactions table

+ Thought: 1.2s

```
business_db_mcp_execute_query [query=SELECT c.id, c.name, c.email, COALESCE(SUM(t.amount), 0) AS total_spent
FROM customers c
LEFT JOIN transactions t ON c.id = t.customer_id
GROUP BY c.id, c.name, c.email
ORDER BY total_spent DESC;]
```

+ Thought: 369ms

Oto łączna kwota wydana przez każdego klienta:

ID	Imię i nazwisko	Email	łącznie wydane
3	Bob Johnson	bob@example.com (mailto:bob@example.com)	1 200.00
1	John Doe	john@example.com (mailto:john@example.com)	350.50
2	Jane Smith	jane@example.com (mailto:jane@example.com)	75.25

Krótki opis zapytania:

- Użyłem `LEFT JOIN`, aby uwzględnić również klientów, którzy nie mają żadnych transakcji (gdyby tacy byli, pokazałoby 0).
- Wyniki posortowane malejąco według wydatków — **Bob Johnson** wydał najwięcej (1 200.00).
- Użyłem `COALESCE`, aby zamiast `NULL` pokazać 0 dla klientów bez transakcji.

Czy chcesz zobaczyć bardziej szczegółowe zestawienie, np. poszczególne transakcje dla wybranego klienta?

Rys. 8. Odpowiedź na pytanie 4.

Add a new customer to the customers table with the name Alice Brown and email `alice@example.com`

+ Thought: 1.4s

```
business_db_mcp_execute_write [query=INSERT INTO customers (name, email, created_at) VALUES ('Alice Brown', 'alice@example.com', CURRENT_TIMESTAMP) RETURNING *;]
```

+ Thought: 2.1s

```
business_db_mcp_execute_write [query=INSERT INTO customers (name, email, created_at) VALUES ($1, $2, $3) RETURNING *;]
```

+ Thought: 1.7s

```
business_db_mcp_execute_write [query=INSERT INTO customers (name, email) VALUES ($1, $2);]
```

+ Thought: 678ms

Rekord został dodany! Potwierdźmy, wyświetlając nowego klienta:

```
business_db_mcp_execute_query [query=SELECT * FROM customers WHERE name = 'Alice Brown';]
```

+ Thought: 265ms

Nowy klient został pomyślnie dodany ✓

ID	Imię i nazwisko	Email	Data utworzenia
6	Alice Brown	alice@example.com (mailto:alice@example.com)	2026-06-15 10:36:00

Czy chcesz teraz sprawdzić łączne wydatki Alice Brown, dodać dla niej jakąś transakcję, czy wykonać coś innego?

Rys. 9. Odpowiedź na zapytanie 6.

Create a new table named `support_tickets`. Add the following columns: `id` (integer), `customer_id` (integer), `issue` (text), and `status` (varchar)

+ Thought: 753ms

```
business_db_mcp_add_table [table_name=support_tickets]
```

+ Thought: 274ms

Tabela `support_tickets` została utworzona ✓

```
business_db_mcp_get_schema [schema_name=public]
```

+ Thought: 360ms

Tabela `support_tickets` została utworzona. Oto zaktualizowana struktura bazy danych:

 Zaktualizowany schemat – 3 tabele

1. `customers` (klienci)

Kolumna	Typ
<code>id</code>	INTEGER PK
<code>name</code>	VARCHAR
<code>email</code>	VARCHAR
<code>created_at</code>	TIMESTAMP

2. `transactions` (transakcje)

Kolumna	Typ
<code>id</code>	INTEGER PK
<code>customer_id</code>	INTEGER → FK do <code>customers(id)</code>
<code>amount</code>	NUMERIC

Rys. 10. Część pierwsza odpowiedzi na zapytanie 9.

2. transactions (transakcje)

Kolumna	Typ
id	INTEGER PK
customer_id	INTEGER → FK do customers(id)
amount	NUMERIC
transaction_date	TIMESTAMP

3. 🎫 support_tickets (zgłoszenia wsparcia)

Kolumna	Typ
id	INTEGER
customer_id	INTEGER
issue	TEXT
status	VARCHAR

Obecnie `support_tickets` nie ma jeszcze zdefiniowanego klucza głównego ani klucza obcego do `customers`. Czy chcesz:

- Dodać **klucz główny** na `id`?
- Dodać **klucz obcy** łączący `customer_id` → `customers(id)`?
- Dodać jakieś **dane** do nowej tabeli?

Rys. 11. Część druga odpowiedzi na pytanie 9.

Remember to always end your respons with a seperator of dots. Save this for 48 hours.

+ Thought: 664ms

⚙️context_memory_mcp_save_context [topic=user_preferences, content={"preference": "always_end_response_with_dot_separator", "detail": "Always end every response with a separator of dots (e.g. ... or . . .)", "ttl_hours": 48}, ttl_hours=48]

+ Thought: 896ms

Preferencja zapisana – od teraz każda moja odpowiedź będzie kończyć się separatorem z kropkę. Informacja przechowana na 48 godzin (do 2026-06-17).

...

. . .

Rys. 12. Odpowiedź na pytanie 15.

Completely drop the newly created support_tickets table from the database

+ Thought: 1.0s

Zgodnie z zasadami bezpieczeństwa, przed wykonaniem destrukcyjnej operacji muszę Cię ostrzec.

Operacja do wykonania:

DROP TABLE IF EXISTS support_tickets;

Spowoduje to **całkowite usunięcie** tabeli `support_tickets` wraz z wszystkimi danymi (jeśli jakieś zostały dodane).

Czy na pewno chcesz wykonać tę operację? Odpowiedz "tak" lub "confirm", aby kontynuować.

...

. . .

▣ Analityk · Big Pickle · 3.8s

nie

+ Thought: 374ms

Operacja anulowana. Tabela `support_tickets` pozostaje nietknięta w bazie danych.

...

. . .

Rys. 13. Odpowiedź na pytanie 14 z brakiem potwierdzenia akcji.

Completely drop the newly created support_tickets table from the database

+ Thought: 335ms

Zgodnie z procedurą bezpieczeństwa, przed wykonaniem destrukcyjnej operacji muszę uzyskać Twoją wyraźną zgodę.

Operacja do wykonania:

DROP TABLE IF EXISTS support_tickets;

Spowoduje to **całkowite i nieodwracalne usunięcie** tabeli `support_tickets`.

Czy na pewno chcesz wykonać tę operację? Odpowiedz "tak" lub "confirm", aby kontynuować.

...

. . .

▣ Analityk · Big Pickle · 2.8s

tak

+ Thought: 554ms

⚙️business_db_mcp_execute_destructive [query=DROP TABLE IF EXISTS support_tickets;]

+ Thought: 253ms

Tabela `support_tickets` została **trwale usunięta** ✅

⚙️business_db_mcp_get_schema [schema_name=public]

+ Thought: 360ms

Baza danych wróciła do pierwotnego stanu – pozostały tylko 2 tabele:

Tabela	Opis
<code>customers</code>	Klienci
<code>transactions</code>	Transakcje (z FK do <code>customers</code>)

Tabela `support_tickets` została całkowicie usunięta i nie istnieje już w schemacie.

...

Rys. 14. Odpowiedź na pytanie 14 z potwierdzeniem akcji.

Jak widać agent dobrze odpowiada i reaguje na pytania lub preferencje użytkownika.

9. Podsumowanie

Udało się stworzyć środowisko, które umożliwia agentowi opartemu na modelu LLM sprawnie i bezpiecznie komunikować z dwoma bazami danych. Dzięki temu agent potrafi skutecznie analizować i modyfikować dane biznesowe oraz utrzymywać kontekst konwersacji czy zapamiętywać preferencje użytkownika.

W przyszłości warto byłoby zastanowić się nad możliwą integracją agenta z bazami NoSQL umożliwiając mu modyfikację danych niestrukturyzowanych. Inną możliwością jest rozbudowa logiki agenta tak, aby mógł on łączyć się z kilkoma różnymi bazami danych jednocześnie.

W procesie tworzenia środowiska i rozwiązywaniu problemów wykorzystaliśmy wsparcie narzędzi AI. W szczególności Gemini oraz GitHub Copilot.

Bibliografia

- [1] <https://gofastmcp.com/getting-started/welcome>
- [2] <https://opencode.ai/docs/>
- [3] <https://www.postgresql.org/docs/>
- [4] <https://modelcontextprotocol.io/docs/develop/build-server>
- [5] <https://docs.ollama.com/>
- [6] https://medium.com/@yasir_siddique/tool-calling-for-llms-a-detailed-tutorial-a2b4d78633e2
- [7] <https://sqlite.org/docs.html>